

SmarTAI 今日工作日志

来源：SmarTAI-shared 目录

2026 年 8 月 28 日

今日概览

类别	条目
今日成果	两问题目拆成两道 满分分配混乱 选择题不应有步骤分
注意事项	题目只按题号致各题型“第 1 题”拥挤 批改速度慢（3 人卷约 1h） 复核建议加“显示答案”
新问题	会被人工答案误导 模型判卷能力不均致低置信度 批改页“运行中”恒为 0

今日成果

今日针对昨日《题目识别与判卷问题记录》中列出的问题完成如下修复。

1 解决昨日问题 1：两问题目拆成两道

1.1 问题背景（昨日记录第 2 节）

同一道解答题（含两问）被重复识别为“整题”与“第一问”两个条目，或一个题目条目内含两道独立的题，导致同一题被识别为两个、题目总数虚高、判卷重复。

1.2 今日处理

- “两问题目”统一拆成两道独立的题目（拆分后各有独立的题号与满分）；
- 拆题后对整个题目集做去重：同一题号、同一题面只保留唯一条目；
- 若题目本身是整题带子问 (1)(2)(3)，则整题作为一个题目，子问作为内部结构，不再单独成条目（与昨日记录的期望一致）；

- 代码佐证：`backend/tools/problem_dedup.py` 的 `dedupe_extracted_problems()` 在分数策略冻结前对抽取行做去重（含“第一问被拆成独立题”场景），并配套回归测试 `backend/tests/test_problem_dedup.py`（22 条原始行 → 19 道题）。

2 解决昨日问题 2：由问题 1 引发的满分分配混乱

2.1 问题背景（昨日记录第 3 节）

重复条目各自带一份满分，导致单题满分与卷面总分不匹配、同一题被重复计分。

2.2 今日处理

- 每道题只存在一个 `max_score`，由教师统一配置；
- 识别阶段不再为重复条目擅自生成满分；拆题去重后满分只落在唯一题目上；
- 总分统计基于去重后的题目集合，不再重复计入；
- 代码佐证：去重发生在 `resolve_question_score_policy()` 冻结满分 之前（`backend/tools/problem_dedup.py` 头注释），保证每题唯一满分（测试 `test_deduped_extraction_feeds_score_policy`）。

3 解决昨日问题 3：选择题不应有步骤分

3.1 问题背景（昨日记录第 1 节）

选择、填空类题目被误判出“过程分”。

3.2 今日处理

- 选择题、填空题只判结果分：正确得满分，错误得 0 分，不再输出过程分项；
- 仅当题干明确要求写出简要过程时，才在题目类型与评分规则中显式启用过程分；
- 代码佐证：`backend/skills/objective.py` 的 `ObjectiveSkill.grade` 明确“result-based, no process scores”，返回 `steps=[]`（客观题永不携带过程分）。

注意事项

1. 环节 3（审核题目）与环节 5（校对作答）中，题目只按题号排列，导致选择题的“第 1 题”、填空题的“第 1 题”、解答题的“第 1 题”挤

在一起，三者共用同一编号难以区分（题号在各题型内重置：单选 1-8、多选 1-3、填空 1-3、解答 1-5）。代码佐证：各列表均按 `number` 数值排序（如 `frontend/app/src/routes/tasks/QuestionPreparationOverviewPage.tsx`、`GradingPreflightPage.tsx` 的 `compareProblems`、`ReviewDetailPage.tsx` 的题目导航）。建议：按“题型 + 题号”分组编号，或在题干前显式标注题型，便于区分。

2. 环节 6（执行批改）速度仍然较慢：3 人份卷子已需约 1 小时。后续可从并发扇出、调用复用、单次调用超时、失败重试/退避等方面继续优化（相关逻辑见 `backend/agents/multi_expert.py`、`backend/agents/grading_agent.py`）。
3. 环节 7（复核批改）建议增加“显示答案”功能：人工复核时能对照参考答案，提高复核效率与准确率。代码佐证：`frontend/app/src/routes/tasks/ReviewDetailPage.tsx` 的复核卡片目前只展示题干、学生作答、评分标准与批改结果，未渲染 `reference_answer`。

新问题

4 会被人工答案误导

4.1 现象

判卷结果可能被人工作答/人工答案误导，从而对学生的试卷给出错误评分。

4.2 解决方案

- (1) 增加 AI 复核环节：由 AI 对批改结果进行二次核对；
- (2) 在注意事项中明确提示，避免人工答案干扰自动批改。

5 不同模型判卷能力不均导致低置信度

5.1 现象

不同模型能力不均衡：千问（Qwen）对部分解答题判不了，DeepSeek（DS）只能判大题。当某题只有部分模型能判时，出现低置信度。

5.2 解决方案

改变“低置信度”的合并机制——只要有一个模型给出高置信度，即视为高置信度结果，不再要求所有模型一致后才判定为高置信度。（现状：

backend/agents/multi_expert.py 中多专家合并采用置信度加权平均，且仅剩单个成功样本时降级为 degraded_to_single 并强制人工复核，见 grading_agent.py 的 _apply_low_confidence_policy。）

6 执行批改页面中的“运行中”始终为 0

6.1 现象

环节 6（执行批改）页面中的“运行中”计数一直显示为 0，即便批改任务实际正在执行中也不同步更新。

6.2 初步定位（代码查证）

- (1) 前端 frontend/app/src/routes/tasks/GradingProgressPage.tsx 的 deriveQueue 用 progress.active 计算“运行中”数量：running = min(total - completed, activePairs.size)，因此只要 active 为空，运行的显示恒为 0；
- (2) 后端 backend/services/task_facade.py 的 _grading_progress() 在任务处于 grading 状态时返回进度对象，其中 "active": [] 是硬编码的空列表；
- (3) async_task_state() 会把内存 reporter 的真实快照（含 active）用上述持久化投影覆盖掉，导致接口返回的 progress.active 恒为空 → 前端“运行中”恒为 0。

6.3 解决方案（建议）

- (1) 在 _grading_progress() 的持久化投影中加入真实的 active 单位（或至少回填 active_units 计数），而不是硬编码空列表；
- (2) 复核 async_task_state() 中内存快照被覆盖的逻辑，避免“运行中”实时信息在接口层丢失；
- (3) 修复后回归验证：批改进行中应显示实际运行题数/人数，完成后归零。

明日计划（建议）

- 按“题型 + 题号”重做环节 3/5 的题目编号显示；
- 继续优化环节 6 批改速度（并发与重试策略）；
- 为环节 7 复核增加“显示答案”功能；
- 落实低置信度合并新机制并回归验证；

- 修复执行批改页面“运行中”始终为 0 的问题。